

ประเมิน 3 รีวิว และ แนวทาง Production สำหรับ PPO_V2 โดย GLM 5.3

คำตอบสั้นก่อน

ผม **เห็นด้วยกับ Claude มากที่สุด** ในแง่การวินิจฉัยโรค วินัยทางสถิติ และการ push back เรื่องเป้าหมายที่ไม่ realistic — และ **เห็นด้วยกับ Gemini มากที่สุด** ในแง่สถาปัตยกรรมที่เอาไป implement ได้จริง (Two-Tier + Vol Targeting คือไอเดียเดียวที่ดีที่สุดที่บรรดาสามร็ว) ส่วน ChatGPT วางกรอบปัญหาได้ตรงที่สุดประโยคเดียว ("ปัญหาคือ Objective Design + Market Representation ไม่ใช่ PPO") แต่ **solution ช่วงกลางของ ChatGPT ผมไม่เห็นด้วย** — เป็น over-engineering ที่ขัดแย้งกับข้อสรุปท้ายของตัวเอง

และมี **2 ประเด็นสำคัญที่ทั้งสามยังไม่จับ** (จะเล่าในหัวข้อ 3)

1. อ่านผลลัพธ์ V2 ให้ตรงก่อน: มัน "ล้มเหลว" อย่างไร

สิ่งที่ควรชมก่อน: engineering ของ V9.2 อยู่ในระดับสูงกว้งงานวิจัยส่วนใหญ่ในวงการ (zero-leakage discipline, open-to-open timeline, no-trade band, friction-matched benchmark, audit suite 8 ข้อ) — ปัญหา **ไม่ใช่ hygiene แต่เป็น problem formulation**

สิ่งที่ตัวเลขบอกจริงๆ:

| หลักฐาน | ความหมาย |

|---|---|

| MaxDD ต่ำกว่า EqW **ทุก fold** | Agent เป็น risk-avoidance agent ที่ทำงานตามที่ reward สอน |

| Excess vs EqW **ติดลบทุก fold** (-5 ถึง -13.7% CAGR) | ไม่มี alpha แม้แต่หน่วยเดียว |

| Sharpe ติดลบใน fold 2, 4 | "การป้องกัน" ของมันไม่ได้เก่งด้วย — defense ที่ไม่มี skill แค่นั่งถือ cash เสียโอกาส |

| AvgCash 20-49% | Cash Trap ตามที่ทุกคนวินิจฉัยถูก |

รากปัญหา 3 ชั้นที่ผมเห็น (เรียงตามความสำคัญ):

1. **Reward สอนให้กลัว**: วันที่ -3% โดนลงโทษรวม ~3 เท่าของวันที่ +3% ได้รางวัล (log-return -3 แล้ว dd penalty อีก -6) แกม `high\_vix\_drag\_multiplier=0.25` ทำให้ **ยิ่ง VIX สูง ยิ่งอนุญาตให้ถือ cash** — สูตรสำเร็จของ Cash Trap โดยตรง

2. **Universe ไม่มี diversifier**: DIA/QQQ/SPY correlate กัน 0.85-0.95 หน ที่ตลาดพังทุกตัวลงพร้อมกัน เครื่องมือเดียวของ agent คือ "หนีไป cash 0%" — alpha ceiling ถูก lock ไว้ตั้งแต่ออกแบบ

3. **Training mechanics** (จะเล่าในหัวข้อ 3 — ไม่มีใครพูดถึง)

2. เห็นตรง / ไม่ตรงกับใคร

ประเด็น	Gemini	ChatGPT	Claude	ผม
Reward redesign เป็นลำดับแรก	☒	☒	☒	☒ เห็นตรงทั้งสาม
ตัด hand-tuned penalties ทั้ง	☒	☒ ยังเก็บ DD penalty	☒ ชัดสุด	☒ ตัดทิ้ง ให้ overlay ทำ
หน้าที่นี่				
Two-tier: RL alpha + deterministic vol targeting	☒ ข้อเสนอที่ดีที่สุด	☒ ใช้ Risk *Agent* แทน		
☒ (risk wrapper)	☒ ทำทันที			
ขยาย universe TLT+GLD	☒	☒ (30-50 ตัว)	☒	☒ แต่ 5-6 ตัวก่อน ไม่ใช่ 50
Multi-Agent RL	—	☒	—	☒ over-engineering
Transformer regime encoder	—	☒	—	☒ ตอนนี้ — ใช้ explicit regime features
สลับเป็น SAC/TD3/DreamerV3	—	☒ (แต่สรุปท้ายถอย)	☒	☒ PPO ไม่ใช่ bottleneck
LSTM/GRU	☒ ทันที	☒	☒ หลังแก้อย่างอื่นก่อน	หลัง Phase 1-2 เท่านั้น
Dirichlet action space	☒	—	—	optional ท้ายๆ
Ensemble หลาย seed	—	—	☒	☒ ถูกและคุ้มมาก
Faber/trend filter เป็น baseline	—	☒	☒ ยกกระดานเป็น "hurdle ที่ RL ต้องข้าม"	
Bootstrap CI / DSR / เพิ่ม fold	☒ DSR/CPCV	☒ ครบสุด	☒ + stitched OOS curve +	
นับ trials				
Push back เรื่อง "Sharpe 1.5-2 ทุก regime"	☒ (บอก impossible แต่ยังโฆษณา 1.5+)	☒ ยืนยัน		
 1.8-2.2 | ☒ | ☒ กับ Claude อย่างเต็มตัว |

****สรุปเชิงรายบุคคล:****

- ****Gemini****: แผนที่จะ implement ได้เร็วที่สุด แกนหลักถูกต้อง แต่โฆษณาเกินจริง 2 จุด — "vol targeting
 เพิ่ม Sharpe 30-50%" เป็น best case จากงาน vol-managed portfolio (ผลจริงแปรผันตามช่วงเวลา และ
 live มักหด 20-30%) และ "DSR พิสูจน์แล้วว่าดัน Sharpe เกิน 1.5" — DSR เป็น objective ที่สมเหตุ
 สมผล (Moody & Saffell) แต่ไม่มี guarantee อย่างนั้น นอกจากนี้ใน code ตัวอย่าง downside penalty มี
 scale เล็กกว่า excess term ~100 เท่า (สเกล scale) และ Dirichlet เป็นสิ่งที่ทำให้เสียเวลาก่อนถึงจุดสำคัญ
 - ****ChatGPT****: diagnosis = ยอดเยี่ยม / solution กลาง = หลงทาง Multi-Agent RL เพิ่ม
 non-stationarity และ credit assignment ปัญหาที่แก้ยาก ทั้งที่ separation นั้นทำได้แบบ deterministic
 (แบบ Gemini) โดยไม่ต้องจ่ายต้นทุนความยากของ MARL; universe 30-50 ตัว + crypto ตอนที่ agent
 ยังแพ้ EqW อยู่เลยคือย่อนลำดับ; และ Phase 7 (เปลี่ยน algorithm) ขัดกับประโยคสรุปของตัวเองที่บอก
 ให้แก้ reward/universe/regime/memory ก่อน
 - ****Claude****: ถูกที่สุดในสิ่งที่ *สำคัญที่สุด* — ความจริงเรื่องเป้าหมาย, สถิติ (Sharpe mean ~0.5 กับ
 SD 0.55-0.59 = ไม่นับสำคัญอะไรเลย), ลำดับการลงมือ (reward → universe → eval คู่ขนาน) และ
 independent risk wrapper — ผมเสริมให้เป็นระบบเต็มรูปแบบข้างล่าง

3. สองจุดที่ทั้งสามรีวิวพลาด (จุดที่ผมเพิ่มเอง)

****3.1 — Episode structure bug: 8 workers เรียนรู้ "วันเดียวกัน" พร้อมกัน****

`reset()` เริ่มที่ `current\_day = 0` เสมอ ทุก episode เริ่มที่ 2000-08-31 และเดิน 14 ปีต่อเนื่อง — ทั้ง 8
 workers ของ `SubprocVecEnv` จึงอยู่ที่ *ช่วงเวลาเดียวกันของประวัติศาสตร์* พร้อมกันตลอด ข้อมูล

8,192 steps ต่อ update ที่ดูเหมือน diverse จริงๆ ซ้ำ regime เดียวกัน 8 เท่า แกรม value function ต้อง predict "ค่าเฉลี่ยของอนาคต" โดย mix regime 2000-2013 ทั้งก่อนจาก local features ที่ซ้ำกัน (RSI 0.2 ในปี 2003 กับ 2011 มี forward return ต่างกันคนละโลก) → vf loss สูง → policy อนุรักษ์นิยม

****การแก้****: ให้แต่ละ worker เริ่มที่ random start day ต่างกัน (+ ความยาว episode สุ่ม เช่น 252-756 วัน) — สิ่งนี้อาจมีผลมากพอๆ กับการ redesign reward เอง

****3.2 — Reward ที่ depend บน hidden state****

`drawdown penalty` ใช้ `peak\_value` และ `prev\_dd` ซึ่ง ****ไม่อยู่ใน observation**** — value function ของ PPO เดาสถานะนี้ไม่ได้ ทำให้ TD target มี variance สูงและ policy หันไปทางปลอดภัยเกินไปทางออก: ตัด penalty นี้ทิ้งทั้งหมด (ให้ deterministic overlay ทำหน้าที่คุม DD) — พอตัดแล้ว reward จะ fully observable 100% ซึ่งเป็นเงื่อนไขที่ value function เรียนรู้ได้จริง

****โบนัส 3 ข้อที่มีคนพูดน้อย****: (ก) cash ใน sim ได้ 0% แต่เงินจริงได้ T-bill yield — ใส่ risk-free accrual จะเปลี่ยน optimal cash behavior ในปี 2022 อย่างมีนัยสำคัญ (ข) คุณอยู่ที่ V9.2 แปลว่ามีอย่างน้อย ~9+ ครั้งของการทดลองล่วงหน้า — ทุก iteration คือ 1 trial ที่กีดกันความน่าเชื่อถือของ OOS Sharpe ต้องนับเข้า Deflated Sharpe (คลาวด์แต่ละประเด็นนี้แต่เบามาก) (ค) test windows ของ 5 fold ต่อเนื่องกัน 2016→2026 — ****ตัวเลขหลักควรเป็น stitched OOS curve เส้นเดียว**** ไม่ใช่ mean ของ 5 fold

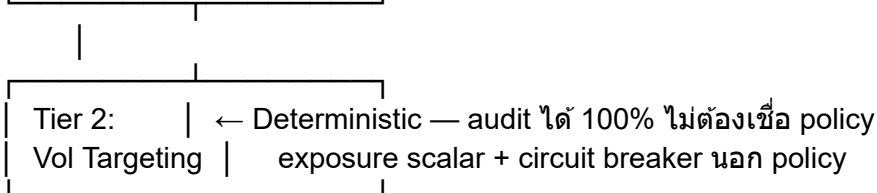
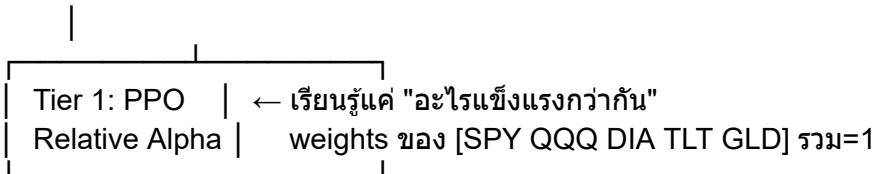
4. แนวทางพัฒนาระดับ Production

4.0 สถาปัตยกรรมเป้าหมาย

...

[Data: vendor จริง + point-in-time store + validation]

|
[Feature Store: train-only scaling, replay-parity ตรวจสอบได้]



|
[Execution: MOC/LOC ที่ Open(t+1) + slippage logging]

|
[Governance: drift monitor, kill-switch, quarterly retrain, model registry]

...

เหตุผลที่แยกแบบนี้ (ตาม Gemini, เสริมด้วยผม): คุณกำลังบังคับให้ RL เรียนรู้ 2 ทักษะพร้อมกัน — allocation และ risk management — แล้วมันเลือกทางที่ง่ายที่สุดคือ "หนี" การแยกสองหน้าที่ทำให้ risk เป็น engineering (น่าเชื่อถือ ตรวจสอบได้) และเหลือให้ RL ทำสิ่งเดียวคือหา alpha

4.1 Phase 0 — Pre-register กติกาความสำเร็จ (ก่อนเขียนโค้ดอีกบรรทัด)

เปลี่ยน "เอาชนะตลาดทุกสภาวะ" ให้เป็นของที่วัดได้ และ**เขียนกติกาตัดสินไว้ก่อนเห็นผล** เพื่อไม่หลอกตัวเองใน loop ของ V9.3, V9.4...:

- Stitched OOS 2016-2026: Sharpe > **ทุก** baseline ทั้ง SPY, EqW, 60/40, **EqW+VolTarget**, **Faber+VolTarget** และ 95% stationary-bootstrap CI ของผลต่างไม่รอบ 0
- MaxDD ≤ 15%, **Up-capture ≥ 0.85, Down-capture ≤ 0.55** (นี่คือนิยามของ "ชนะทุกสภาวะ" ที่ยั่งยืนได้จริง)
- DSR ≥ 0.95 โดยนับ trials จาก experiment log ทั้งหมด

4.2 Phase 1 — แก้ learning problem (ใช้ข้อมูลเดิม, ~1-2 สัปดาห์)

1. **Reward v3**: ตัด `dd\_penalty`, `cash\_drag`, `high\_vix\_drag` ทั้งหมด

...

$$r_t = 100 \times (r_{\text{agent},t} - r_{\text{bench},t}) - \lambda_{\text{to}} \times \text{turnover}_t$$

...

โดย `r\_bench` = EqW ของ risky universe คิด cost และ timeline เดียวกัน (สร้างใน env ได้ทันทีเพราะเห็น open matrices เดียวกัน — แบบ Gemini ทำ) และ λ_{to} เล็กมาก ทดสอบ 3 variants ควบคู่กัน: (A) excess return (B) DSR (C) excess/rolling-vol — เลือกด้วย **validation เท่านั้น ห้ามแตะ test**

2. **Random-start ต่อ worker** + ความยาว episode สุ่ม (แก้ correlated rollouts — หัวข้อ 3.1)
3. **เพิ่ม cash yield** ตาม fed funds effective (point-in-time จาก FRED)
4. **Ensemble 3-5 seeds** เผลี่ย action
5. สร้าง baselines ให้ครบ: EqW, SPY, 60/40, **EqW-VT, Faber-VT**

****Gate 1****: bull folds (2016-17, 2020-21) มี avg invested ≥ 85%; stitched OOS ≥ EqW; excess ≥ 0 อย่างน้อย 3/5 folds — ถ้าไม่ผ่าน แปลว่าปัญหาอยู่ที่ representation ต้องแก้ก่อนไปต่อ

4.3 Phase 2 — Universe + Deterministic Risk Overlay (~2-4 สัปดาห์)

1. Universe = **SPY, QQQ, DIA, TLT, GLD** (TLT = ballast ดอน equity ร่วง, GLD = stagflation hedge — ไม่ใช่แค่ "เห็น feature แต่เทรตไม่ได้" แบบปัจจุบัน)
2. **Vol targeting overlay**: $\sigma_t = \text{EWMA}(\lambda=0.94)$ ของ daily returns ของ basket ใช้ถึง Close(t) เท่านั้น → $s_t = \text{clip}(\sigma_{\text{daily}} / \sigma_t, 0.25, 1.0)$ → $w_{\text{final}} = [1-s_t, s_t \cdot w_{\text{agent}}]$ พร้อม hysteresis (ปรับเฉพาะเมื่อ scalar เปลี่ยน > 5%) กัน churn จากตัว scalar เอง และ **คิด transaction cost ของ overlay จริง**

3. **Circuit breaker นอก policy**: portfolio DD เกิน 10% → force exposure ≤ 50% จนกว่ากลับมาใกล้ peak

4. อย่างข้าม ablation: รัน (ก) EqW+VT ล้วน (ข) Faber+VT ล้วน (ค) ระบบเต็ม — **alpha ของ RL มีความหมายก็ต่อเมื่อ (ค) > (ก),(ข) อย่างมีนัยสำคัญ** ไม่อย่างนั้นคุณจะได้ผลของ overlay เป็นผลของ RL (กับดักที่ Gemini ไม่ได้เตือน)

Gate 2: stitched Sharpe ≥ 1.2, MaxDD ≤ 15%, ชนะ EqW-VT ที่ CI 95%, และยังเหลือ edge ที่ cost 30bps — ถ้าไม่ผ่านสองรอบการทดลอง → หยุดเพิ่มความซับซ้อน และ productionize แบบ rules-based (Faber-VT) ก่อน โดย RL เป็น research track ต่อ **นี้ไม่ใช่ความล้มเหลว** — Faber-VT บน 5 assets เป็น production strategy ที่สมเหตุสมผลมาก

4.4 Phase 3 — Representation upgrades (มีเงื่อนไขเท่านั้น)

- Frame-stack 10-20 วันของ features/returns ก่อน — ถ้าช่วย ค่อยขึ้น `RecurrentPPO` (sb3-contrib) — **ไม่ใช่ก่อน** และไม่ใช่ transformer
- Regime features แบบ explicit และ point-in-time: yield curve slope (10y-3m), VIX/VIX3M, ระยะห่างจาก SMA200, vol ratio สั้น/ยาว
- Cross-sectional: relative momentum, ratio QQQ/SPY ฯลฯ (ตาม Gemini — จุดที่ถูกต้อง)
- Dirichlet head = experiment ปิดท้าย ถ้ามีเวลาเหลือ

4.5 Phase 4 — Statistical hardening

- Stationary bootstrap CI ทุก metric หลักบน stitched curve; **DSR** โดยนับ N_{trials} จาก log รวมทุกเวอร์ชันตั้งแต่ V1
- CPCV เป็นรอง (ระวัง leakage ตอน implement), cost stress 30/50bps, execution-delay stress
- ตารางแยก regime (bull/bear/chop) + up/down capture ต่อ fold

4.6 Phase 5 — Production engineering

- Data**: ออกจาก yfinance — vendor จริง หรือ store ที่ controlled + validation (gap/outlier checks); **feature parity test**: replay ข้อมูลย้อนหลัง 1 เดือนผ่าน live pipeline ต้องได้ features ตรงกับตอน train 100% (สิ่งนี้จับ lookahead bug ที่ backtest จับไม่ได้)
- Inference**: batch หลังปิดตลาด → target weights → orders ก่อน open (MOC/LOC สำหรับขนาด ~\$1M ใน ETF หลัก liquidity สูง พอ) + slippage logging เทียบสมมติฐาน
- Risk ops นอก policy**: kill-switch (live DD -12% → halt + human review; rolling 60d live Sharpe หลุด CI ของ backtest → halt), feature drift (PSI), action distribution drift
- Retrain governance**: quarterly walk-forward retrain, champion/challenger, model registry (hash ของ code+data+config), rollback plan
- Paper trading 3-6 เดือน** ด้วย gate เบื้องต้น: live-vs-sim daily correlation ≥ 0.95, slippage อยู่ในระดับ → เงินจริงขนาดเล็ก → scale ทีละขั้น

5. ความจริงที่ต้องยอมรับเรื่องเป้าหมาย

ผมต้องพูดตรงๆ ตามที่ Claude พูดและผมเห็นด้วยอย่างเต็มตัว: *****Sharpe 1.5-2 + MaxDD ต่ำสุด + ชนะตลาดทุกสภาวะ**" สำหรับ long-only ETF รายวัน เป็น spec ที่ขัดกันเอง 2 ข้อ** ถ้าถือ cash เพื่อลด DD คุณจะแพ้ bull หันที่ — ยกเว้นมี alpha จริง ซึ่งเป็นสิ่งที่ต้องพิสูจน์ผ่าน Gate 2-3 ไม่ใช่สิ่งที่ตั้งเป้าแล้วไปให้ถึง

Scale ความเป็นไปได้ (จากโครงสร้าง universe สะสมทีละขั้น):

ระบบ	Sharpe ที่ realistic
3 correlated equities + cash (V2 ปัจจุบัน)	ceiling ~0.8-1.0 แม้ทำอะไรกับ reward
+ TLT/GLD (cross-asset)	~1.0-1.3
+ vol targeting overlay	~1.2-1.6 (backtest), ~0.9-1.4 live
+ tactical alpha ที่ ***จริง***	>1.5 — แต่ต้องผ่าน ablation พิสูจน์ก่อน

ถ้าต้องการ absolute return สูงพร้อม Sharpe สูง ทางเลือกที่ mathematically ถูกต้องคือ ****leverage 1.2-1.5× บน core ที่ Sharpe สูง**** (แลกกับ margin/financing cost ที่ต้อง model จริง) — ไม่ใช่หวังให้ long-only ชนะทุกอย่างพร้อมกัน

****สิ่งที่ควรทำสัปดาห์นี้****: (1) เปลี่ยน reward + ลบ penalties ทั้งสามออก (2) random-start ต่อ worker (3) รัน 5 folds ด้วย ensemble 3 seeds แล้วดูแค่ 3 ตัวเลข: avg invested ใน bull folds, stitched OOS Sharpe, excess vs EqW (4) สร้าง EqW-VT และ Faber-VT ให้เสร็จ — เพราะถ้าไม่มีสองตัวนี้ คุณจะไม่มีทางรู้เลยว่า RL ของคุณมีค่าหรือไม่มีค่า (5) เปิด experiment log และเขียน pre-registered gates ตาม Phase 0 — คุณอยู่ใน loop iteration มาแล้วอย่างน้อย 9 รอบ และสิ่งที่กัน overfitting ได้ดีที่สุดไม่ใช่โค้ดที่ฉลาดขึ้น แต่คือวินัยในการตัดสินใจว่า ***เมื่อไหร่ควรหยุด***

